

SIMPLEX Central Operations Console

USER GUIDE

Client Quickstart & Installation Manual (Windows, Linux & macOS)

1-Click Docker Deployment

SIMPLEX Central Operations Console

Production Deployment, Hosting & Client Operations Manual

Official Multi-Platform Guide (Windows, Linux & macOS) · Version 1.0.0

Executive Summary

The **SIMPLEX Central Operations Console** is an enterprise automation, multi-client credential orchestration, and desktop agent management system. This document serves as the authoritative operational manual for system administrators, IT engineers, and enterprise clients deploying and maintaining the application.

The system is delivered as a pre-compiled, containerized software bundle using **Docker**. End-users never interact with raw source code, development runtimes, or complex build toolchains.



1. Choosing Your Deployment Topology

The application supports two primary enterprise deployment topologies depending on whether your host machine already runs Microsoft SQL Server:

Topology A: Standalone Docker Host (Turnkey Automatic Installation)

- **When to choose:** The server or host machine does NOT have Microsoft SQL Server installed.
- **How it works:** Docker runs the official Microsoft SQL Server 2022 container alongside the API and Web containers.
- **Footprint:** ~2 GB RAM, 100% turnkey. The user simply executes `start.bat` or `./start.sh`.

Topology B: Shared Host with Existing Microsoft SQL Server (Enterprise Mode)

- **When to choose:** The server already has an active Microsoft SQL Server instance hosting other enterprise or clinical databases.
- **How it works:** Docker runs ONLY the API and Web Console containers (~300 MB RAM total). The containers connect to the host's existing SQL Server via `host.docker.internal:1433`.
- **Zero Database Contamination Guarantee:** Our automated database provisioning script applies strict `DENY VIEW ANY DATABASE` rules. The SIMPLEX application has **ZERO visibility** into any other corporate or clinical databases on that server.

2. Database Security & Isolation Architecture

When deploying onto an existing SQL Server instance containing sensitive databases (e.g. Hospital Information Systems, Payroll, Clinical Records), strict database-level sandboxing is enforced.

The Automated Sandboxing Script (`init-db.sql`)

Run the following idempotent provisioning script once in **SQL Server Management Studio (SSMS)** or via `sqlcmd`:

```

-- =====
-- SIMPLEX Central Operations Console – Database Isolation & Sandboxing
-- =====

USE [master];
GO

-- 1. Create Dedicated Application Database (if not exists)
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = N'SIMPLEX_CENTRAL_DB')
BEGIN
    PRINT 'Creating dedicated database [SIMPLEX_CENTRAL_DB]...';
    CREATE DATABASE [SIMPLEX_CENTRAL_DB];
END
GO

-- 2. Create Dedicated Sandboxed Login
IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name = N'simplex_app_user')
BEGIN
    PRINT 'Creating isolated login [simplex_app_user]...';
    CREATE LOGIN [simplex_app_user]
    WITH PASSWORD = N'SimplexApp@Secure2026!',
        CHECK_POLICY = OFF,
        CHECK_EXPIRATION = OFF;
END
GO

-- 3. Grant Control ONLY on SIMPLEX_CENTRAL_DB
USE [SIMPLEX_CENTRAL_DB];
GO
IF NOT EXISTS (SELECT name FROM sys.database_principals WHERE name = N'simplex_app_user')
BEGIN
    CREATE USER [simplex_app_user] FOR LOGIN [simplex_app_user];
    ALTER ROLE [db_owner] ADD MEMBER [simplex_app_user];
END
GO

-- 4. CRITICAL SECURITY RULE: Deny visibility to all other databases on this server!
USE [master];
GO
PRINT 'Enforcing strict database sandboxing...';
DENY VIEW ANY DATABASE TO [simplex_app_user];
GO

PRINT '✓ SIMPLEX Database and Sandboxed Login Provisioned Successfully!';
GO

```

Security Guarantee:

- **Zero Cross-Database Visibility:** When `simplex_app_user` queries `sys.databases`, only `SIMPLEX_CENTRAL_DB` and `master` appear. All other existing clinical/enterprise databases remain completely invisible.
- **Permission Rejection:** Even if an unauthorized query attempts `SELECT * FROM HospitalBilling.dbo.Invoices`, SQL Server strictly terminates the request with a permission denied error.

3. Production File Specifications

The distribution package contains clean, human-readable operational orchestration files:

3.1 Production Environment Configuration (`.env`)

```
# =====  
# SIMPLEX Central Operations Console – Production Environment  
# =====  
  
# Database Connection Settings  
# For Topology A (Docker DB): Use MSSQL_HOST=simplex_db  
# For Topology B (Existing DB): Use MSSQL_HOST=host.docker.internal  
MSSQL_HOST=simplex_db  
MSSQL_PORT=1433  
MSSQL_DATABASE=SIMPLEX_CENTRAL_DB  
MSSQL_USER=sa  
MSSQL_PASSWORD=Rajesh@123  
  
# Service Ports  
API_PORT=3000  
WEB_PORT=5173  
  
# Pre-Built Official Container Images (GitHub Container Registry)  
SIMPLEX_API_IMAGE=ghcr.io/rajeshsupport/simplex/simplex-api:latest  
SIMPLEX_WEB_IMAGE=ghcr.io/rajeshsupport/simplex/simplex-web:latest  
  
# Cryptographic Master Keys  
ENCRYPTION_MASTER_KEY=e8b839655f46a7be7e3c15c6b7582b1c853f6517a942bcba5e7e600d89e574ac  
JWT_SECRET=simplex_jwt_secret_production_2026_super_secure_token  
JWT_REFRESH_SECRET=simplex_jwt_refresh_production_2026_super_secure_token  
AGENT_SHARED_SECRET=simplex_agent_shared_key_2026
```

3.2 Orchestration File: Topology A (Standalone with Database)

File: `docker-compose.yml`

```

version: '3.8'

services:
  # 1. Dedicated MSSQL Database Container
  simplex_db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: simplex_mssql_db
    restart: always
    environment:
      ACCEPT_EULA: "Y"
      SA_PASSWORD: "${MSSQL_PASSWORD:-Rajesh@123}"
      MSSQL_PID: "Developer"
    ports:
      - "${MSSQL_PORT:-1433}:1433"
    volumes:
      - simplex_mssql_data:/var/opt/mssql

  # 2. Central API Backend (Pre-compiled Container)
  simplex_api:
    image: ${SIMPLEX_API_IMAGE:-ghcr.io/rajeshsupport/simplex/simplex-api:latest}
    container_name: simplex_central_api
    restart: always
    ports:
      - "${API_PORT:-3000}:3000"
    environment:
      MSSQL_HOST: simplex_db
      MSSQL_PORT: 1433
      MSSQL_DATABASE: ${MSSQL_DATABASE:-SIMPLEX_CENTRAL_DB}
      MSSQL_USER: sa
      MSSQL_PASSWORD: "${MSSQL_PASSWORD:-Rajesh@123}"
      MSSQL_ENCRYPT: "false"
      MSSQL_TRUST_SERVER_CERTIFICATE: "true"
      PORT: 3000
      NODE_ENV: production
      API_BASE_URL: http://localhost:${API_PORT:-3000}
      WEB_BASE_URL: http://localhost:${WEB_PORT:-5173}
      ENCRYPTION_MASTER_KEY: "${ENCRYPTION_MASTER_KEY}"
      JWT_SECRET: "${JWT_SECRET}"
      JWT_REFRESH_SECRET: "${JWT_REFRESH_SECRET}"
      AGENT_SHARED_SECRET: "${AGENT_SHARED_SECRET}"
    depends_on:
      - simplex_db

  # 3. Web Operations Console (Pre-built React SPA via Nginx)
  simplex_web:
    image: ${SIMPLEX_WEB_IMAGE:-ghcr.io/rajeshsupport/simplex/simplex-web:latest}
    container_name: simplex_central_web
    restart: always
    ports:
      - "${WEB_PORT:-5173}:80"
    depends_on:
      - simplex_api

volumes:
  simplex_mssql_data:
    name: simplex_production_db_data

```

3.3 Orchestration File: Topology B (Host with Existing MSSQL)

File: `docker-compose.existing-mssql.yml`

```

version: '3.8'

services:
  # 1. Central API Backend (Connects directly to Host's Existing SQL Server)
  simplex_api:
    image: ${SIMPLEX_API_IMAGE:-ghcr.io/rajeshsupport/simplex/simplex-api:latest}
    container_name: simplex_central_api
    restart: always
    ports:
      - "${API_PORT:-3000}:3000"
    extra_hosts:
      - "host.docker.internal:host-gateway"
    environment:
      MSSQL_HOST: host.docker.internal
      MSSQL_PORT: ${MSSQL_PORT:-1433}
      MSSQL_DATABASE: ${MSSQL_DATABASE:-SIMPLEX_CENTRAL_DB}
      MSSQL_USER: ${MSSQL_USER:-simplex_app_user}
      MSSQL_PASSWORD: "${MSSQL_PASSWORD:-SimplexApp@Secure2026!}"
      MSSQL_ENCRYPT: "false"
      MSSQL_TRUST_SERVER_CERTIFICATE: "true"
      PORT: 3000
      NODE_ENV: production
      API_BASE_URL: http://localhost:${API_PORT:-3000}
      WEB_BASE_URL: http://localhost:${WEB_PORT:-5173}
      ENCRYPTION_MASTER_KEY: "${ENCRYPTION_MASTER_KEY}"
      JWT_SECRET: "${JWT_SECRET}"
      JWT_REFRESH_SECRET: "${JWT_REFRESH_SECRET}"
      AGENT_SHARED_SECRET: "${AGENT_SHARED_SECRET}"

  # 2. Web Operations Console
  simplex_web:
    image: ${SIMPLEX_WEB_IMAGE:-ghcr.io/rajeshsupport/simplex/simplex-web:latest}
    container_name: simplex_central_web
    restart: always
    ports:
      - "${WEB_PORT:-5173}:80"
    depends_on:
      - simplex_api

```

4. How Clients Obtain the Delivery ZIP Package

The delivery package `simplex-delivery.zip` is intentionally ultra-lightweight (~10 KB) because it contains only the operational launchers and configurations.

Option 1: Direct Vendor Sharing (Standard Enterprise Method)

- The software vendor provides `simplex-delivery.zip` via **Email attachment**, **Secure Cloud Link (Google Drive / OneDrive)**, or **USB Pen Drive**.
- Save and extract the ZIP file to your preferred directory (e.g. `C:\SIMPLEX` on Windows or `/opt/simplex` on Linux).

Option 2: Direct Download via GitHub Releases

- Visit the official release repository: 📄 <https://github.com/rajeshsupport/SIMPLEX/releases>
- Click on `simplex-delivery.zip` to download directly.

- Or download via command line:

```
curl -L -O https://github.com/rajeshsupport/SIMPLEX/raw/main/simplex-delivery.zip
```

5. Operating System Step-by-Step Installation Guides

5.1 WINDOWS Guide (Windows 10 / 11 / Windows Server)

Step 1: Install Docker Desktop for Windows

1. Download **Docker Desktop for Windows**: 🖱️ <https://www.docker.com/products/docker-desktop/>
2. Run `Docker Desktop Installer.exe`. Ensure the "Use WSL 2" option is enabled.
3. Restart your computer if prompted.
4. Launch Docker Desktop and wait until the whale icon in the bottom-right taskbar turns green with "Engine running".

Step 2: Extract the Package

1. Right-click `simplex-delivery.zip` and select **Extract All...**
2. Select destination folder (e.g., `C:\SIMPLEX`).

Step 3: Launch the Application (1-Click)

1. Double-click on `start.bat`.
2. The command prompt will automatically:
 - Verify Docker health.
 - Download pre-built images from GitHub Container Registry.
 - Launch Database, API, and Web Console.
 - Automatically open your default web browser to: 🖱️ <http://localhost:5173>

Step 4: Maintenance Operations

- **To Stop:** Double-click `stop.bat`.
- **To Apply Updates:** Double-click `update.bat` (Downloads new patches in 15 seconds; database records are 100% preserved).

5.2 LINUX Guide (Ubuntu 22.04 / 24.04, Debian, RHEL)

Step 1: Install Docker on Linux

Execute the standard official Docker installation:

```
# Update repositories and install dependencies
sudo apt-get update && sudo apt-get install -y ca-certificates curl gnupg

# Add Docker GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/doc
ker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Apt repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.do
cker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine and Compose Plugin
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Grant user privileges
sudo usermod -aG docker $USER
```

Step 2: Extract and Launch

```
unzip simplex-delivery.zip -d ~/simplex
cd ~/simplex
chmod +x *.sh
./start.sh
```

Open your browser and navigate to: 🖱️ <http://localhost:5173> (or http://<SERVER_IP>:5173)

Step 3: Maintenance Operations

- To Stop: `./stop.sh`
- To Apply Updates: `./update.sh`

5.3 macOS Guide (Apple Silicon M1/M2/M3/M4 & Intel)

Step 1: Install Docker Desktop for Mac

1. Download **Docker Desktop for Mac**: 🖱️ <https://www.docker.com/products/docker-desktop/>
 - Select **"Mac with Apple Silicon"** for M-series chips.
 - Select **"Mac with Intel chip"** for Intel Macs.
2. Drag Docker to your **Applications** folder and start it.
3. Verify the whale icon in the top menu bar says **"Docker Desktop is running"**.

Step 2: Extract and Launch

1. Double-click `simplex-delivery.zip` to extract.
2. Open Terminal (`Cmd + Space` -> type `Terminal` -> Enter):

```
cd ~/Downloads/simplex-delivery
chmod +x *.sh
./start.sh
```

3. Open web browser to: <http://localhost:5173>

6. Access Endpoints & Default Credentials

Component	Network Endpoint	Default Credentials	Description
Web Operations Portal	http://localhost:5173	Username: admin Password: Rajesh@123	Main operational dashboard.
Central REST API	http://localhost:3000/api/v1	Bearer Token / Session	Orchestration and scheduling engine.
Swagger API Docs	http://localhost:3000/api/docs	Public / Authorize Header	Interactive API documentation.
Database Server	localhost:1433	Host: simplex_db / Port: 1433 DB: SIMPLEX_CENTRAL_DB	Microsoft SQL Server 2022 instance.

[!NOTE] Upon your initial login, navigate to **Settings** -> **Security** to change the default administrator password.

7. Over-The-Air (OTA) Updates with Zero Data Loss

Whenever the software vendor publishes bug fixes or enhancements:

1. The vendor pushes verified code to GitHub.
2. GitHub Actions automatically builds and tags the new public Docker image (ghcr.io/rajeshsupport/simplex/...:latest).
3. The client simply executes:
 - **Windows:** Double-click [update.bat](#)
 - **Linux/Mac:** Run [./update.sh](#)
4. Docker pulls the updated container layers and restarts the services within 15–30 seconds.
5. **Data Protection Guarantee:** The database volume ([simplex_production_db_data](#)) is decoupled from application containers. Registered clients, hospital credentials, users, and audit histories are **never modified or lost** during updates.

8. Frequently Asked Questions (FAQ)

Q1: Our server already has Microsoft SQL Server. Will there be port conflicts?

- **Answer:** If you use **Topology B**, the Docker database container is completely disabled. The API connects directly to your existing SQL Server over port 1433 via `host.docker.internal`. Zero port conflicts occur.

Q2: Can multiple hospital staff members access the portal across our LAN?

- **Answer:** Yes. Obtain the local IP address of the host machine (e.g. `192.168.1.100`). Any authorized computer on the hospital Wi-Fi or LAN can access the web console via `http://192.168.1.100:5173`.

Q3: What if port 5173 or 3000 is occupied by another local service?

- **Answer:** Open `.env` in any text editor. Modify `WEB_PORT=8080` and `API_PORT=3001`. Save and run `start.bat` (or `./start.sh`). The portal will then serve on `http://localhost:8080`.

SIMPLEX Central Operations Console · Enterprise Deployment Manual · Copyright © 2026. All rights reserved.